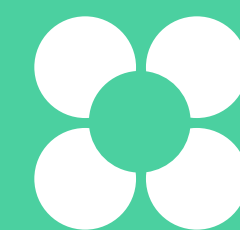


Полиморфизм



Проблема

Зачастую возникают ситуации, когда необходимо описать несколько объектов, которые имеют общие черты, но отличаются в поведении.

Примерами могут быть:

животные — все умеют подавать голос, но каждое делает это по-своему:

- корова мычит
- собака лает

транспортные средства — все умеют перемещаться, но:

- корабль плывёт
- машина едет

Фигуры

Рассмотрим задачу.

Мы начинаем делать аналог AutoCAD – программы для чертежей.

На самом раннем этапе мы реализуем примитивы: линия, треугольник, прямоугольник, круг.

Нам необходимо иметь возможность посчитать площадь, периметр, а также задавать цвет каждой из фигур

Процедурный подход

Определять тип фигуры и выбирать соответствующую формулу:

```
double getArea(Figure figure) {  
    if (CIRCLE.equals(figure.getType())) {  
        //  $\pi * R * R$   
    } else if (RECTANGLE.equals(figure.getType())) {  
        //  $a * b$   
    } else if ...  
}
```

ООП-подход

Для этого предлагаю создать иерархию классов:

- базовый класс Figure
- Line
- Triangle
- Rectangle
- Circle

Заглушка

```
class Color {  
    private String name;  
    public Color(String name) {  
        this.name = name;  
    }  
    public String getName() { return name; }  
}
```

Color — класс из фреймворка Android. На данный момент создадим для него заглушку, хранящую только имя цвета

Цвет фигуры

Он присущ всем рассматриваемым фигурам, поэтому разместим его в базовом классе:

```
class Figure {  
    private Color fillColor;  
    public Figure(Color fillColor) {  
        this.fillColor = fillColor;  
    }  
  
    public Color getFillColor() {  
        return fillColor;  
    }  
    ...  
}
```

Периметр и площадь

Периметр и площадь мы можем рассчитать для каждой из фигур.

Проблема:

- функции должны быть объявлены в базовом классе, так как присущи всем объектам
- для разных объектов функции должны работать по-разному

Понятие полиморфизма

Дословный перевод: поли + морфизм = много форм
(с русского на русский).

Это способность объекта выглядеть, как его предок,
но отличаться в поведении

Периметр и площадь

Объявим необходимые методы в базовом классе **Figure**:

```
class Figure {  
    public Color getFillColor() {  
        ...  
    }  
    public double getPerimeter() {  
        throw new NotImplementedException();  
    }  
    public double getArea() {  
        throw new NotImplementedException();  
    }  
}
```

Здесь мы выбрасываем исключения, так как не можем реализовать их в этом классе. На следующем занятии мы увидим, как это можно сделать красивее и надёжнее

Triangle

Создадим класс треугольника:

```
class Triangle extends Figure {  
    public Triangle(Color fillColor) {  
        super(fillColor);  
    }  
    ...  
}
```

Как нам хранить положение треугольника?
Как хранить положение других фигур?

Point

Создадим вспомогательный класс Point:

```
class Point {  
    private int x, y;  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    public int getX() { return x; }  
    public int getY() { return y; }  
    public static double getDistance(Point p1, Point p2) {  
        return Math.sqrt(  
            Math.pow(p2.getX() - p1.getX(), 2)  
            + Math.pow(p2.getY() - p1.getY(), 2)  
        );  
    }  
}
```

Triangle

Ключевое слово `@Override` означает, что мы переопределяем поведение унаследованной от родителя функции

Point

```
class Triangle extends Figure {  
  
    private Point a, b, c; // вершины треугольника  
    // todo-конструктор, принимающий цвета + три вершины  
  
    @Override  
    public double getPerimeter() {  
        return Point.getDistance(a, b)  
            + Point.getDistance(b, c)  
            + Point.getDistance(c, a);  
    }  
  
    @Override  
    public double getArea() {  
        // формула площади треугольника по трём сторонам  
        double hp = getPerimeter() / 2.0;  
        return Math.sqrt(  
            hp  
            * (hp - Point.getDistance(a, b))  
            * (hp - Point.getDistance(b, c))  
            * (hp - Point.getDistance(c, a))  
        );  
    }  
}
```


Использование

```
static double totalPerimeter = 0.0;
static double totalArea = 0.0;
// Функция принимает базовый класс!
static void addFigure(Figure f) {
    totalPerimeter += f.getPerimeter();
    totalArea += f.getArea();
}
static void main(String[] args) {
    addFigure(new Triangle(...));
    addFigure(new Rectangle(...));
    addFigure(new Circle(...));
    System.out.format("Perimeter %.2f\n", totalPerimeter);
    System.out.format("Area %.2f", totalArea);
}
```

Обратите внимание, мы не знаем, какая фигура будет передана. Её ещё может даже не существовать на момент компиляции этого кода, если мы делаем библиотеку, которую впоследствии отдадим другим разработчикам

Сужение и расширение типа

```
static void main(String[] args) {  
    // Расширение типа  
    // Присвоение сработает  
    // Явное приведение типов не требуется  
    Figure figure = new Triangle(...);  
  
    // Сужение типа  
    // Присвоение сработает  
    // Явное приведение типов требуется  
    Triangle triangle = (Triangle) figure;  
  
    // Сужение типа  
    // Присвоение не сработает, т. к. объект в переменной – треугольник  
    // Явное приведение типов требуется  
    Rectangle rectangle = (Rectangle) figure;  
}
```


Сужение и расширение типа

```
static void main(String[] args) {  
    // Расширение типа  
    // Присвоение сработает  
    // Явное приведение типов не требуется  
    Figure figure = new Triangle(...);  
  
    // Вызвать можем только то, что есть у Figure  
    // Но вызванный будет работать в версии из Triangle  
    figure.someMethod();  
}
```

Доступ к переопределённым функциям

Представим, что нам понадобилось создать полупрозрачный треугольник. Мы могли бы сделать это двумя способами:

- 1 Изменить значение в конструкторе при создании:

```
class TransparentTriangle {  
    public TransparentTriangle(Color fillColor) {  
        // обращаемся к конструктору класса-родителя  
        super(new Color("полупрозрачный" + fillColor.getName()));  
    }  
}
```

Доступ к переопределённым функциям

② Частично изменив поведение родительской функции:

```
class TransparentTriangle {  
    @Override  
    public Color getFillColor() {  
        // обращаемся к методу класса-родителя  
        return new Color("полупрозрачный" + super.getFillColor().getName());  
    };  
}
```

Обратите внимание на ключевое слово `super` — именно оно позволяет получить доступ к «предыдущей версии» метода